

Gymnázium, Praha 6, Arabská 14
předmět Programování, vyučující Mgr. Jan Lána

Plošinová hra „Útěk z ostrova“

ročníkový projekt



Poděkování

Rádi bychom poděkovali panu profesoru Mgr. Janu Lánovi, který byl vedoucí naše ročníkové práce. Děkujeme mu za pomoc a zpětnou vazbu během vypracování tématu, rady a nápady k obsahu práce.

Anotace

Ve své práci se snažíme o vytvoření vypečené plošinové hry jménem „Útěk z ostrova“. Práce zahrnuje využití dovedností v oboru programování, zejména v jazyku C++, v počítačové pixelové grafice, propojení obou částí a uvedení hry do funkce. Dokumentace obsahuje popis procesu vytvoření hry a její jednotlivých složek, vysvětlení jak hra funguje, ukázky samotné hry a kódu.

Abstract

In this project, we aim to create a polished platformer game titled "Escape from the Island." The project involves applying skills in programming, particularly in C++, in computer pixel graphics, integrating both components, and bringing the game to life. The documentation includes a description of the game development process and its individual components, an explanation of how the game works, and examples of the game itself and the code.

Anmerkung

In unserer Arbeit versuchen wir, ein ausgefeiltes Plattformspiel mit dem Titel „Flucht von der Insel“ zu entwickeln. Die Arbeit umfasst die Anwendung von Programmierkenntnissen, insbesondere in der Sprache C++, sowie Kenntnisse in der computergestützten Pixelgrafik, die Verknüpfung beider Bereiche und die Inbetriebnahme des Spiels. Die Dokumentation enthält eine Beschreibung des Entstehungsprozesses des Spiels und seiner einzelnen Komponenten, eine Erläuterung der Funktionsweise des Spiels sowie Beispiele für das Spiel selbst und den Code.

Résumé

Dans ce projet, nous nous efforçons de créer un jeu de plateforme bien ficelé intitulé « Évasion de l'île ». Ce travail fait appel à des compétences en programmation, notamment en langage C++, en infographie pixelisée, à l'intégration de ces deux aspects et à la mise en service du jeu. La documentation comprend une description du processus de création du jeu et de ses différents composants, une explication de son fonctionnement, ainsi que des exemples du jeu et du code.

Prohlášení

Prohlašujeme, že jsme jediní autoři tohoto projektu, všechny citace jsou řádně označené a všechna použitá literatura a další zdroje jsou v práci uvedené. Tímto dle zákona 121/2000 Sb. (tzv. Autorský zákon) ve znění pozdějších předpisů udělujeme bezúplatně škole Gymnázium, Praha 6, Arabská 14 oprávnění k výkonu práva na rozmnožování díla (§ 13) a práva na sdělování díla veřejnosti (§ 18) na dobu časově neomezenou a bez omezení územního rozsahu.

V Praze, dne 3. 5. 2026

Obsah

Úvod.....	6
1. Gameplay.....	7
2. Návrh a struktura hry.....	8
2.1 Hlavní komponenty systému.....	8
2.2 Tok programu.....	9
3. Programování hry.....	10
3.1 Použité technologie.....	10
3.2 Herní smyčka.....	11
3.3 Multithreading.....	11
3.4 Herní logika.....	12
3.5 Kolize a interakce.....	12
4. Grafika.....	13
4.1 Vizuální motiv.....	13
4.2 Proces vytváření grafiky.....	14
4.3 Propojení grafiky s kódem.....	14
5. Uvedení hry do funkce.....	15
6. Jak nainstalovat?.....	16
7. Možná vylepšení.....	17
Závěr.....	18
Zdroje.....	19

Úvod

Hra „Útěk z ostrova“ je singleplayerová hra typu plošinovka. Hra je vytvořena ve 2D prostředí a využívá pixelovou grafiku. Hra obsahuje jednu úroveň, jejíž obtížnost se postupně zvyšuje během postupování hráče. Cílem hráče je dostat se ze začátku úrovně do cíle a překonat všechny překážky, které se na cestě objeví. Úroveň se bude postupně ztěžovat – budou se objevovat nové překážky a herní mechaniky, jakmile hráč postupuje dál. Hráč si tak bude muset osvojit logiku hry, zapamatovat si rozmístění překážek a najít způsob, jak úroveň úspěšně projít.

1. Gameplay

Hra je 2D plošinovka kde se hráč se snaží uniknout z podzemního prostředí opuštěného ostrova postupným překonáváním jednotlivých levelů.

Základním principem hry je pohyb postavy v prostředí s překážkami. Hráč musí správně načasovat pohyb, skoky a reakce na herní prvky, aby se dostal bezpečně na druhý konec úrovně. Mezi hlavní herní mechaniky patří:

- **pohyb do stran (vlevo a vpravo)**
- **skoky**
- **vyhýbání se nástrahám**

Úrovně jsou navrženy tak, aby představovaly výzvu. Na začátku jsou úrovně jednodušší a slouží k seznámení s ovládáním a základními principy hry. Postupně se obtížnost zvyšuje a hráč je nucen reagovat rychleji a přesněji.

Cílem hráče je projít jednotlivé levely a dostat se na jejich konec bez ztráty života nebo pádu do nebezpečných zón. Úspěšné dokončení levelu umožňuje postup do další části hry. Cílem celé hry je dostat se ven z podzemí kde hra končí.

2. Návrh a struktura hry

Architektura hry „Útěk z ostrova“ je navržena modulárně s důrazem na oddělení jednotlivých částí systému a jejich vzájemnou spolupráci. Celý projekt je postaven na vlastních datových strukturách, označovaných jako „*clustery*“, které umožňují přehlednou a efektivní správu herních objektů, grafiky i renderovacích prostředků. Tento přístup usnadňuje rozšiřitelnost programu a zároveň zlepšuje organizaci kódu.

Základním prvkem je struktura *WRCluster*, která obsahuje ukazatel na hlavní okno aplikace a dynamické pole rendererů. Díky tomu je možné pracovat s více renderovacími kontexty a flexibilně řídit vykreslování. Další klíčovou komponentou je *ECluster*, který slouží jako kontejner pro všechny *entity* ve hře. Tento kontejner uchovává objekty reprezentující hráče, pozadí i jednotlivé překážky.

Každá entita je reprezentována strukturou *entity*, která obsahuje informace o své pozici, velikosti, pohybu, životě a dalších vlastnostech. Součástí *entity* je také struktura *TCluster*, která spravuje její textury. *TCluster* umožňuje organizovat textury do podskupin, například pro různé animace, a dynamicky mezi nimi přepínat. Tento mechanismus je klíčový pro animaci postav a dalších herních prvků.

Zpracování vstupu od uživatele je řešeno pomocí struktury *user_events*, která převádí události z knihovny *SDL* na jednoduché logické hodnoty. Díky tomu je následná práce s vstupy přehlednější a oddělená od samotné herní logiky.

Celý program probíhá v několika na sebe navazujících fázích. Nejprve dojde k inicializaci knihoven a vytvoření okna, následně k načtení konfigurace, inicializaci entit a textur a poté ke spuštění hlavní herní smyčky. Po ukončení hry jsou všechny prostředky korektně uvolněny.

2.1 Hlavní komponenty systému

Hra je postavena na několika základních komponentách, které společně tvoří její architekturu. Tyto komponenty jsou navrženy tak, aby byly co nejvíce nezávislé, což umožňuje jejich snadnou údržbu a případné rozšíření.

Jednou z klíčových částí je struktura *WRCluster*, která zajišťuje správu hlavního okna a renderovacích nástrojů. Obsahuje ukazatel na okno aplikace a kolekci rendererů, které jsou využívány při vykreslování jednotlivých prvků hry. Na tuto komponentu navazuje systém správy entit, reprezentovaný strukturou *ECluster*, která slouží jako kontejner pro všechny herní objekty.

Každý herní objekt je reprezentován strukturou *entity*. Ta obsahuje nejen informace o své pozici a pohybu, ale také další vlastnosti, jako jsou životy, poškození nebo stavové příznaky. Důležitou součástí *entity* je také struktura *TCluster*, která zajišťuje správu textur a jejich animaci. Tato struktura umožňuje organizovat textury do skupin a efektivně mezi nimi přepínat.

Zpracování vstupu od uživatele je řešeno pomocí struktury *user_events*, která převádí události z knihovny *SDL* na jednodušší reprezentaci vhodnou pro další zpracování. Další významnou komponentou je systém konfigurace, který načítá hodnoty z externího souboru a umožňuje tak měnit chování programu bez nutnosti úprav zdrojového kódu.

Celý systém doplňuje logovací mechanismus, který zaznamenává důležité události během běhu programu, a modul pro ošetření chyb, který zajišťuje bezpečné ukončení aplikace v případě problému.

2.2 Tok programu

Tok programu popisuje jednotlivé kroky, které aplikace vykonává od svého spuštění až po ukončení. Tento proces je důležitý pro pochopení toho, jak spolu jednotlivé části systému spolupracují v čase a jak na sebe navazují.

Po spuštění programu dochází nejprve k inicializaci základních systémů. To zahrnuje vytvoření logovacího souboru, inicializaci knihovny *SDL* a nastavení potřebných subsystémů pro práci s grafikou a vstupy. Následně je vytvořeno hlavní okno aplikace spolu s rendererem, který bude zajišťovat vykreslování.

Dalším krokem je načtení konfiguračního souboru, ze kterého program získá důležité parametry, jako jsou rozměry okna, škálování grafiky nebo cesty k souborům s texturami. Na základě těchto informací jsou inicializovány jednotlivé entity a připraveny jejich datové struktury.

Po inicializaci entit dochází k načtení grafických dat. Program postupně načte jednotlivé obrázky, upraví jejich velikost podle nastavení a vytvoří z nich textury, které jsou následně přiřazeny ke konkrétním objektům ve hře.

Jakmile jsou všechny komponenty připraveny, je spuštěna hlavní herní smyčka. Ta zajišťuje nepřetržitý běh hry, zpracování vstupů, aktualizaci herního stavu a vykreslování scény. Současně s hlavní smyčkou mohou běžet další vlákna, která se starají například o animace nebo pohyb objektů.

Po ukončení herní smyčky, například při zavření okna, program přechází do fáze ukončení. V této fázi jsou bezpečně ukončena všechna běžící vlákna, uvolněny všechny alokované prostředky, zničeno okno i renderer a následně je ukončena knihovna *SDL*. Tím je zajištěno, že program skončí korektně bez úniku paměti nebo jiných problémů.

3. Programování hry

Hra byla implementována v programovacím jazyce C++ s využitím knihoven *SDL3* a *SDL3_image*, které zajišťují práci s grafikou a vstupy. Při návrhu programu byly využity moderní prvky jazyka C++, zejména práce s vlákny, atomickými proměnnými a synchronizačními mechanismy.

Základem běhu programu je herní smyčka, která je implementována ve funkci *MainLoop*. Tato smyčka běží v hlavním vlákne a opakovaně provádí tři hlavní operace: zpracování vstupu od uživatele, aktualizaci herního stavu a vykreslení aktuální scény. Mezi jednotlivými iteracemi je vloženo krátké zpoždění, které omezuje počet vykreslených snímků za sekundu a stabilizuje běh programu.

Zajímavým prvkem projektu je využití více vláken. Kromě hlavního vlákna jsou vytvořena samostatná vlákna pro hráče a pro pozadí. Vláknem hráče zajišťuje jeho pohyb a animaci, zatímco vlákno pozadí se stará o animaci prostředí. Tato vlákna běží paralelně a komunikují s hlavním vlákem pomocí sdílených dat. Pro zajištění bezpečného přístupu k těmto datům jsou použity mutexy, které zabráňují současnému zápisu do kritických částí paměti.

Herní logika je oddělena do samostatného prostoru jmen *GameConditions*. Například pohyb hráče je řízen funkcí *PlayerMovement*, která reaguje na vstupy z klávesnice a podle nich upravuje směr pohybu i aktuální animaci postavy. Tento přístup umožňuje snadné rozšiřování logiky bez nutnosti zásahů do ostatních částí programu.

Kolizní systém je založen na práci s obdélníkovými hitboxy, které jsou reprezentovány strukturou *SDL_FRect*. Kolize se detekují porovnáním pozic a rozměrů těchto hitboxů. I když je současná implementace spíše základní, poskytuje dostatečný základ pro další rozšiřování, například o přesnější fyziku nebo komplexnější interakce mezi objekty.

3.1 Použité technologie

Při vývoji hry byly využity moderní nástroje a technologie, které umožňují efektivní tvorbu interaktivních aplikací. Základním programovacím jazykem je C++, který poskytuje vysoký výkon a zároveň umožňuje využití pokročilých konceptů, jako je objektově orientované programování nebo práce s vlákny.

Pro práci s grafikou a vstupy byla použita knihovna *SDL3* (*Simple DirectMedia Layer*), která zajišťuje komunikaci s hardwarem a umožňuje vytváření oken, zpracování vstupu a vykreslování grafiky. Pro načítání obrázků ve formátu *PNG* byla využita rozšiřující knihovna *SDL3_image*.

Důležitou součástí projektu je také využití standardní knihovny jazyka C++, zejména pro práci s vlákny (*std::thread*), synchronizaci (*std::mutex*, *std::atomic*) a práci se souborovým systémem (*std::filesystem*). Tyto nástroje umožňují implementaci paralelního zpracování a efektivní správu dat.

Projekt dále využívá externí konfigurační soubor, který je načítán při spuštění programu a umožňuje upravovat parametry aplikace bez zásahu do zdrojového kódu. Pro ladění a sledování průběhu programu je implementován vlastní logovací systém, který zaznamenává důležité události do souboru.

3.2 Herní smyčka

Hlavní herní smyčka představuje základní mechanismus, který řídí běh celé aplikace. Je implementována ve funkci *MainLoop* a běží v hlavním vlákne programu. Smyčka se opakuje po celou dobu běhu hry a v každé iteraci provádí několik klíčových operací.

Nejprve dochází ke zpracování vstupu od uživatele pomocí struktury *user_events*, která aktualizuje stav jednotlivých vstupních proměnných. Následně je na základě těchto vstupů aplikována herní logika, například pohyb hráče. Poté je vykreslena aktuální herní scéna pomocí rendereru. Na konci každé iterace je vloženo krátké zpoždění, které omezuje rychlost běhu smyčky a zabraňuje nadměrnému zatížení procesoru.

Tento cyklus se opakuje, dokud uživatel nepožádá o ukončení programu, například zavřením okna.

3.3 Multithreading

Program využívá více vláken pro oddělení různých částí výpočtu a zlepšení plynulosti běhu hry. Kromě hlavního vlákna, které se stará o zpracování vstupu a vykreslování, jsou vytvořena další vlákna pro animaci a pohyb objektů.

Vlákno hráče je zodpovědné za aktualizaci jeho pozice a současně za přepínání textur, což vytváří animaci pohybu. Podobně vlákno pozadí zajišťuje animaci prostředí, například pohyb vody. Tato vlákna běží paralelně s hlavním vláknem a neustále aktualizují stav příslušných objektů.

Aby nedocházelo ke konfliktům při současném přístupu k datům, jsou využity synchronizační mechanismy, zejména mutexy. Ty zajišťují, že v jeden okamžik může kritickou část kódu vykonávat pouze jedno vlákno. Ukončení vláken je řízeno pomocí atomické proměnné, která signalizuje, kdy mají vlákna bezpečně skončit.

3.4 Herní logika

Herní logika je oddělena od ostatních částí programu a je implementována v prostoru jmen *GameConditions*. Tento přístup umožňuje přehledné uspořádání kódu a usnadňuje jeho rozšiřování.

Jedním z hlavních prvků logiky je řízení pohybu hráče. Funkce *PlayerMovement* reaguje na vstupy z klávesnice a podle nich nastavuje směr pohybu, rychlost a odpovídající animaci. Například při stisknutí klávesy pro pohyb doleva se změní směr pohybu hráče a zároveň se aktivuje odpovídající sada textur.

Pokud hráč nevykonává žádný pohyb, logika zajistí přepnutí do klidového stavu, což se projeví změnou animace. Tento systém umožňuje plynulé přechody mezi jednotlivými stavy a tvoří základ pro další rozšiřování herních mechanik.

3.5 Kolize a interakce

Kolizní systém je založen na porovnávání obdélníkových hitboxů, které reprezentují jednotlivé entity. Každá entita má definovaný svůj hitbox pomocí struktury *SDL_FRect*, která obsahuje informace o pozici a velikosti objektu.

Detekce kolize probíhá kontrolou, zda se hitboxy dvou objektů překrývají. Pokud dojde ke kolizi, může na jejím základě dojít k různým herním reakcím, například k odebrání životů nebo zastavení pohybu. Implementace také obsahuje základ pro práci se životy a poškozením, kdy jedna entita může ovlivnit stav jiné entity.

Současná verze systému je spíše základní, ale je navržena tak, aby ji bylo možné v budoucnu rozšířit o složitější fyzikální chování nebo přesnější detekci kolizí.

4. Grafika

Tato kapitola se zabývá vysvětlením a nastíněním grafické části celé hry. Grafika hry je nezbytnou součástí nejen našeho projektu, ale i každé počítačové hry. Má za cíl zapůsobit na hráče, zpříjemnit herní zkušenost a dodat hře estetickou a uměleckou složku.

Grafika se dělí na pixelovou (rastrovou) a vektorovou. V této ročníkové práci používáme pixelovou grafiku, to znamená, že obraz je tvořen za pomoci jednotlivých pixelů (čtverečků), každý z nich jde zvlášť obarvit. Daný typ grafiky jsme vybrali, protože se používá pro tvorbu her, složitějších obrázků (lokace, postavy apod.). Grafika celé hry byla vytvořena v aplikaci Pixel studio a celá je zpracována ve stylu pixel art.

4.1 Vizuální motiv

Práce zahrnuje návrh a tvorbu herních levelů, postav a překážek. Cílem bylo vytvořit vizuální styl odpovídající prostředí opuštěného podzemí ostrova a také scénérii samotného ostrova.

Celá hra je navržena v jednodušším 2D stylu. Tento přístup byl zvolen především kvůli přehlednosti během hraní a také kvůli menší technické náročnosti a zároveň je vizuálně atraktivní. Barevná paleta se skládá primárně z barev, které mají přírodní objekty, tedy primárně zelená, šedá a hnědá.

Hra využívá objekty typické pro prostředí opuštěného ostrova, jako jsou kameny, opuštěné doly, tunely nebo stalagmity. Nástrahy a překážky jsou tvořeny právě těmito přírodními prvky, čímž je zachována jednotná atmosféra celého herního prostředí.

Finální lokace hry, což je zároveň cíl hráče, byla vymyšlená a inspirovaná fantasy filmy, skutečnými rostlinami z botanické zahrady v Krakově a trochu kulturou pirátů. Před hráčem se objevuje krajina s mořem, pláží, lodí, lítajícím ostrovem a tropickými horami. Dominantní barvy jsou modrá a její odstíny, olivová, hořčicová a oranžová. Krajina symbolizuje svobodu, začátek něčeho nového, budoucnost.



[Obrázek 1] Ukázka pozadí levelu.

4.2 Proces vytváření grafiky

Proces tvorby grafiky probíhal v několika fázích, od prvotního návrhu až po finální implementaci do hry. Po stanovení jasného vizuálního motivu a stylu hry byly vytvořené jednoduché návrhy postav a levelů. Také byly vytvořeny náčrtky ostrova pro lepší představu místa a následně jednodušší realizaci v aplikaci Pixel Studio. Náčrtky jsem vytvářela nejdříve rukou a potom i v programu. Po fázi náčrtku jsem přidávala více konkrétních detailů na obraz ostrova. Pro lepší a přesnější vykreslení elementů lokace, jsem používala geometrické útvary, jakožto obdélník, čára, kruh.

Při práci s pixelart grafikou bylo využíváno škálování (upscaling), kdy byly menší obrázky (např. 32×32 px) zvětšovány na vyšší rozlišení (např. 128×128 px). Byla použita metoda „nearest-neighbor“, která zachovává ostré hrany a typický vzhled pixel artu bez rozmazání.

Po vytvoření základních verzí grafiky následovalo jejich testování přímo ve hře. Na základě zpětné vazby ze hry byla upravována například:

- velikost objektů
- barevnost
- čitelnost postav a překážek
- rozlišení

4.3 Propojení grafiky s kódem

Grafická část hry je úzce propojena s programovým kódem prostřednictvím systému dynamického načítání textur. Veškerá grafika je uložena v externích souborech, jejichž názvy a struktura jsou definovány v konfiguračních souborech. Program tyto soubory při spuštění načítá a na jejich základě vytváří textury použitelné pro vykreslování.

Proces načítání začíná otevřením *PNG* souboru, který je následně převeden na *SDL_surface*. Tento surface je poté škálován podle nastaveného koeficientu, přičemž je použita metoda nearest-neighbor, která zachovává ostré hrany typické pro pixel art. Nakonec je vytvořena textura, kterou lze přímo vykreslit na obrazovku.

Textury jsou organizovány pomocí struktury *TCluster*, která umožňuje jejich rozdělení do logických skupin. Tyto skupiny odpovídají například různým animacím postavy, jako je chůze doleva nebo doprava. Aktivní textura se v průběhu hry mění, což vytváří efekt animace. Tato změna probíhá buď v hlavním vlákne, nebo v samostatných vláknech určených pro animaci.

5. Uvedení hry do funkce

Spuštění programu probíhá prostřednictvím funkce *WinMain*, která slouží jako vstupní bod aplikace. Na začátku je vytvořen logovací soubor, do kterého jsou zapisovány důležité informace o běhu programu. Následně dojde k inicializaci knihovny *SDL*, bez které by nebylo možné pracovat s grafikou ani vstupy.

Po úspěšné inicializaci je vytvořeno hlavní okno aplikace a renderer, který zajišťuje vykreslování. Program poté načte konfigurační soubor, ze kterého získá potřebné informace o nastavení hry, například rozměry okna nebo umístění grafických souborů. Následuje inicializace všech entit a načtení jejich textur.

Jakmile jsou všechny komponenty připraveny, je spuštěna hlavní herní smyčka. Po ukončení této smyčky, například při zavření okna, program korektně ukončí všechna vlákna, zničí renderer i okno a uvolní všechny použité prostředky.

6. Jak nainstalovat?

Pro spuštění hry je nejprve nutné zajistit, aby byly všechny potřebné soubory umístěny ve správné složce. Jedná se zejména o spustitelný soubor programu, konfigurační soubor a složku obsahující grafické assety. Pokud je projekt distribuován jako archiv, je potřeba jej nejprve rozbalit.

Po přípravě všech souborů stačí spustit výsledný „*exe*“ soubor. Program si při spuštění automaticky načte potřebná data a inicializuje herní prostředí. Pro správný běh je vyžadován operační systém Windows a podpora knihoven *SDL3* a *SDL3_image*. Dále je nutná funkční grafická karta, která umožňuje vykreslování pomocí GPU.

7. Možná vylepšení

Do budoucna by bylo možné projekt dále rozšířit například o pokročilejší fyzikální systém, který by přesněji řešil kolize a pohyb objektů. Dalším krokem by mohlo být přidání více herních stavů, jako je hlavní menu nebo pauza, případně implementace zvukových efektů a hudby. Vylepšit by bylo možné také synchronizaci vláken a optimalizaci renderování. Z dlouhodobého hlediska by bylo zajímavé vytvořit editor levelů nebo rozšířit hru o multiplayer.

Závěr

Cílem této práce bylo vytvořit funkční 2D plošinovou hru s využitím programovacího jazyka C++ a knihovny *SDL*. Na rozdíl od jednodušších projektů byl kladen důraz na návrh vlastní architektury, která odděluje jednotlivé části programu a umožňuje jejich nezávislý vývoj.

Během práce jsme se seznámili s pokročilejšími koncepty, jako je multithreading, synchronizace vláken nebo dynamické načítání dat z konfiguračních souborů. Projekt tak nepředstavuje pouze jednoduchou hru, ale také základ pro vlastní herní engine, který je možné dále rozšiřovat.

Získané zkušenosti lze využít nejen při dalším vývoji her, ale i v jiných oblastech programování, kde je potřeba pracovat s komplexnější strukturou programu a efektivním řízením zdrojů.

Zdroje

SDL3 dokumentace. [online] [3. 5. 2026]. Dostupné z < <https://wiki.libsdl.org/SDL3/FrontPage>

>

SDL_image dokumentace. [online] [3. 5. 2026]. Dostupné z <

https://wiki.libsdl.org/SDL3_image/FrontPage >

C++ reference. [online] [3. 5. 2026]. Dostupné z < <https://cppreference.com/> >

vlastní zdrojové kódy. [online] [3. 5. 2026]. Dostupné z < https://github.com/gyarab/2025-3e-utek_z_ostrova >

Seznam obrázků, grafů a tabulek

Obrázek 1 – Ukázka pozadí levelu. [str. 11]

Použité technologie

- Vývoj softwaru:
 - Microsoft Visual Studio 2022 – jazyk C++
- Pracovní software: